MODELS OF COMPUTATION | TEAM PROJECT

A Mealy Machine Model of the Enigma Rotor

Complete study guide: every state, every transition, the interactive diagram, implementation details.

26	26
STATES (q0 TO q25)	INPUT SYMBOLS (A–Z)
26	**676**
OUTPUT SYMBOLS (A–Z)	TRANSITIONS
q0	**0**
START STATE	ACCEPTING STATES

What this document is

Every number, transition and trace shown across this guide comes directly from the machine data file (`enigmaData.js`) and its pure execution simulator, ensuring this study guide cannot drift from the underlying mathematical machine.

Study Guide Contents

§1 The One-Minute Explanation

§2 How the System Works End to End

§3 Formal Definition

§4 Complete Mealy Machine Diagram

§5 Transition Table and Matrix

§6 State-by-State Details

§7 What the Machine Computes

§8 Simulator and Worked Traces

§9 Theory Points a Teacher May Probe

§10 Implementation Details

Team: Harsha Narayan J, Lingeshwar Rethinam, Sanjay, Pavan Aadarsh

Course: Models of Computation (Research Paper Review)

Section 1: The One-Minute Explanation

Say this first

We model one Enigma rotor as a Mealy machine where each of the 26 states represents a physical rotational rotor position. Every key press outputs a scrambled letter and immediately advances the rotor to the next position. Crucially, the output character depends on both the current state and the specific key pressed.

Four things the machine demonstrates

DEMONSTRATION	ONE-SENTENCE MEANING	LIVE DATA TRACE
1. Same key, different output	Pressing the exact same key repeatedly yields varying cipher outputs because the rotor advances each time.	`Input: "AAA" from q0 → Output: "JKC" (States: q0 → q1 → q2 → q3)`
2. Rotor stepping	Every keystroke deterministically advances the machine state from position `qN` to `q(N+1)`.	`Input: "A" from q0 → Next State: q1 (Rule: qN → q(N+1))`
3. Full turn	After exactly 26 keystrokes, the rotor completes a full 360-degree cycle and returns back to state `q0`.	`Input: 26 × "A" from q0 → Final State: q0 (26 steps = 1 complete revolution)`
4. Wrap-around	Stepping is strictly modular: advancing from the final rotational position `q25` wraps back around to `q0`.	`Input: "AA" from q25 → Output: "EJ" (States: q25 → q0 → q1)`

Be upfront about this

This model implements the internal scrambling permutation and mechanical stepping of a single rotor only. It has no reflector and no plugboard, so it is not a full Enigma and does not by itself encrypt in a reversible reciprocal way. Historical cryptanalysis methods (the Turing-Welchman Bombe, clonking, Banburismus) are analyzed in our research paper review, but are not modeled by this finite state machine.

Why a Mealy machine and not a lookup table or if-statements?

A static program or simple lookup table cannot capture the computational properties of an Enigma rotor:

- **State-dependent output:** The emitted cipher letter is a function of both current rotor position (state) AND input character ($\lambda: Q \times \Sigma \to \Gamma$).
- **State memory:** The machine maintains historical context across key presses; a flat lookup table has no internal memory.
- **Strict determinism:** For any given state and input letter, exactly one transition path and one output character exist.
- **Linear time execution:** Processing n keystrokes takes O(n) time via constant-time O(1) table lookups per keystroke.
- **Explainable trace:** The formal state trajectory demonstrates precisely how each ciphertext character was physically generated.
- **Formal rigor:** As a standard theoretical automaton, it can be mathematically analyzed, proved complete, and tested for state equivalence and minimality.

Section 2: How the System Works End to End

The system operates as a finite state transducer with a clear sequential pipeline:

Plain Text
Raw keystrokes from operator

↓

Clean Input
Validated uppercase letters (A–Z)

↓

Machine Run
One transition move per character

↓

Output & Trace
Ciphertext and visited state path

Worked Example: Input "ABC" from Start State q0

STEP	CURRENT STATE	LETTER READ	OUTPUT LETTER	NEXT STATE
1	`q0`	`A`	`J`	`q1`

STEP	CURRENT STATE	LETTER READ	OUTPUT LETTER	NEXT STATE
2	q1	B	D	q2
3	q2	C	D	q3

Result: Input "ABC" from q0 produces output "JDD", ending in state q3.

Where this sits in the paper's method

STAGE IN PAPER	WHAT THE RESEARCH PAPER DOES	COVERED BY OUR MACHINE?
Stage 1: Finding the plugboard	Constructs cribs, menus, Turing Bombe hardware, and checking machine circuits to deduce plugboard stecker pairings.	No (our model has no plugboard and no bombe search)
Stage 2: Army and Air Force	Searches turnover point and clonking over 26 × 26 = 676 ring-setting combinations.	No (our model has no notches or ring settings)
Stage 2: Navy	Uses Banburismus and the German Navy EINS catalog over 26^3 = 17,576 window positions to identify rotor positions.	No (our model uses rotational states q0 to q25 with no window letters or catalog searches)
Rotor stepping as a state machine	Formal deterministic state transition model capturing internal pin wiring permutations and rotational stepping per keystroke.	Yes (what we built)

Honest scope

Our machine is a formal theoretical model of one rotor's stepping and scrambling mechanisms, not an implementation of the Bombe or cryptanalytic attacks. If asked in an examination whether this project breaks Enigma, the correct and precise answer is that **it models the foundational physical machine that the cryptanalysts attacked**.

Section 3: Formal Definition

In formal language theory, a **Mealy machine** is a 6-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, \lambda, q0)$$

TUPLE ELEMENT	FORMAL NAME	DEFINITION IN OUR ENIGMA ROTOR MODEL
`Q`	Finite set of states	26 states `{q0, q1, ..., q25}`, corresponding to the 26 rotational positions of the rotor.
`Σ`	Input alphabet	26 uppercase Latin alphabet symbols `{A, B, ..., Z}` representing keys pressed on the keyboard.
`Γ`	Output alphabet	26 uppercase Latin alphabet symbols `{A, B, ..., Z}` representing illuminated lampboard letters.
`δ`	Transition function	Total function `Q × Σ → Q` (676 entries). Advances the rotor cyclically: $\delta(q_N, \sigma) = q_{(N+1) \bmod 26}$.
`λ`	Output function	Total function `Q × Σ → Γ` (676 entries). In each state q_N, $\lambda(q_N, \cdot)$ is a strict permutation of A–Z.
`q0`	Start state	State `q0` (the initial reference position of the rotor).
`F`	Accepting states	**None** (empty set ∅). A Mealy machine is a finite-state transducer that reads a finite input string and emits one output letter per input letter, with no accepting states. A short run, including an empty one, is valid.

Alphabet Chips (Interactive Letter View)

The input and output alphabets Σ and Γ share the 26 standard uppercase characters. **Click any letter chip** to toggle Letter View on the Mealy machine diagram below:

From one rotor to the full machine

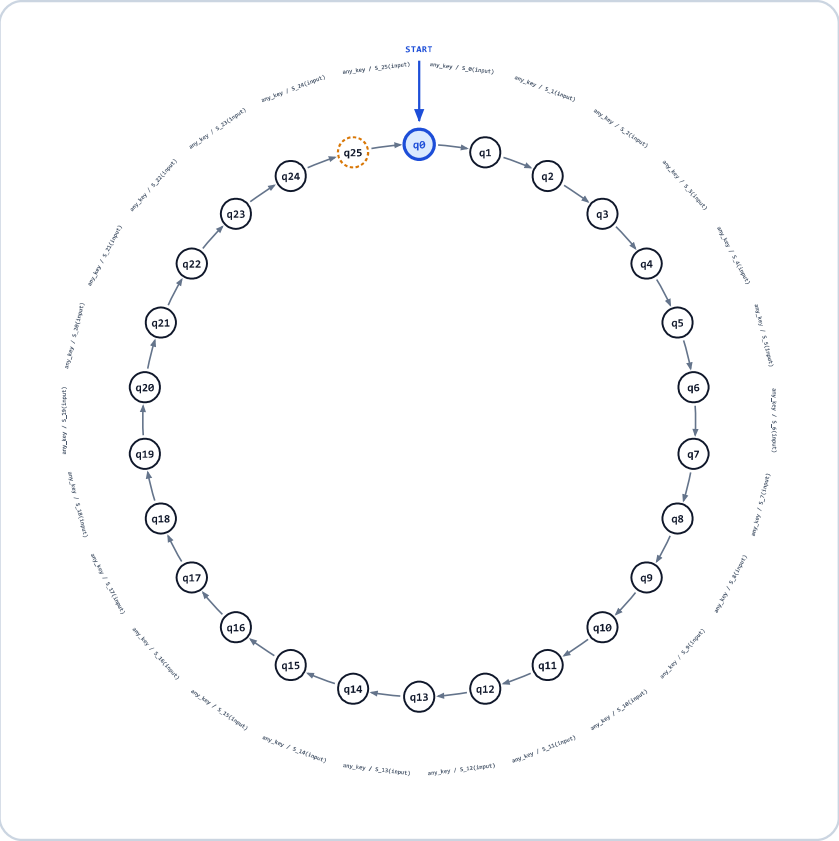
Three unconstrained 26-contact rotors yield 26 × 26 × 26 = 17,576 theoretical window positions. However, due to the middle rotor's mechanical ratchet double-stepping anomaly, only 26 × 25 × 26 = 16,900 states are physically reachable during continuous encryption (as documented in the paper review's addendum). Our project models a single rotor in isolation, which has exactly 26 rotational states (q0 to q25), with no window letters, no notch positions, and no ring settings.

Mealy Machine vs. Deterministic Finite Automaton (DFA)

DIMENSION	MEALY MACHINE (OUR ROTOR)	STANDARD DFA (ACCEPTOR)
Primary Purpose	Transducer: converts an input string into an output string.	Classifier: decides membership in a formal language ($w \in L$).
Output Mechanism	Emits an output symbol on every state transition ($\lambda: Q \times \Sigma \to \Gamma$).	No transition outputs; emits only an Accept/Reject verdict upon string termination.
Accepting States	No accepting states ($F = \emptyset$); every valid input character is processed.	Designated set of final accepting states $F \subseteq Q$.
State Incompleteness	Must be total (all 676 transitions defined) to encrypt all inputs.	Can be partial, often implicitly routing undefined transitions to a dead state.

Section 4: Complete Mealy Machine Diagram

Complete transition diagram of the single-rotor transducer. Exactly 26 states (`q0–q25`) arranged cyclically, with 26 composite ring edges representing all 676 input-level substitutions.



26 states — 26 ring edges (each carrying 26 inputs) — 676 input-level transitions — complete Mealy machine — no accepting states

Scrambling table (state q0)

Wiring permutation at current rotor position

INPUT	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
OUTPUT	J	L	E	K	F	C	P	U	Y	M	S	N	V	X	G	W	T	R	O	Z	H	A	Q	B	I	D
NEXT	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1	q1

State q0 Complete Profile

Meaning: Rotor position 0 (offset +0) **Reached From:** q25 **Goes To:** q1 **Fixed Point:** R → R

ALL 26 INPUT/OUTPUT PAIRS AT THIS POSITION:

A → J	B → L	C → E	D → K	E → F	F → C	G → P	H → U	I → Y
J → M	K → S	L → N	M → V	N → X	O → G	P → W	Q → T	R → R ★
S → O	T → Z	U → H	V → A	W → Q	X → B	Y → I	Z → D	

Diagram Symbols & Notation Legend

- **Start State (q0):** Initial rotor position. Thicker solid outline with light tinted fill and an incoming "start" pointer arrow. *(No double-ring is used, confirming this transducer has no accepting states.)*
- **Normal States (q1–q24):** Intermediate rotational positions. Single solid outline with white fill.
- **Wrap-Around State (q25):** Terminal rotor position. Dashed outline; its transition cycles back to `q0`.
- `any_key` : Any keyboard character (A–Z) causing mechanical rotation to state `q(n+1)`.
- `S_n(input)` : Scrambler output character determined by internal pin wiring at position `qn`.
- **Transducer Model:** *No accepting states* (no double-circle states). A finite-state transducer that reads a finite input string and emits one output letter per input letter, with no accepting states. A short run, including an empty one, is valid.

Section 5: Transition Table and Matrix

5a. Transition Matrix (26 States × 26 Letters)

Because every key press deterministically advances the rotor by one rotational position, the next state is identical across all 26 inputs for any given row and is displayed in the dedicated "Next state" column. Each cell shows the output character produced by that state for the corresponding input key.

Legend: ★ **Fixed-point cell:** The internal rotor scrambling maps this input letter directly to itself (output equals input).

• Hover to inspect row/column • Click to pin • Tab and arrow keys to navigate.

STATE	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	NEXT STATE
q0	J	L	E	K	F	C	P	U	Y	M	S	N	V	X	G	W	T	R★	O	Z	H	A	Q	B	I	D	q1
q1	K	D	J	E	B	O	T	X	L	R	M	U	W	F	V	S	Q★	N	Y	G	Z	P	A	H	C	I	q2
q2	C	I	D	A	N	S	W	K	Q	L	T	V	E	U	R	P★	M	X	F	Y	O	Z	G	B	H	J	q3
q3	H	C	Z	M	R	V	J	P	K	S	U	D	T	Q	O★	L	W	E	X	N	Y	F	A	G	I	B	q4
q4	B	Y	L	Q	U	I	O	J	R	T	C	S	P	N★	K	V	D	W	M	X	E	Z	F	H	A	G	q5
q5	X	K	P	T	H	N	I	Q	S	B	R	O	M★	J	U	C	V	L	W	D	Y	E	G	Z	F	A	q6
q6	J	O	S	G	M	H	P	R	A	Q	N	L★	I	T	B	U	K	V	C	X	D	F	Y	E	Z	W	q7
q7	N	R	F	L	G	O	Q	Z	P	M	K★	H	S	A	T	J	U	B	W	C	E	X	D	Y	V	I	q8
q8	Q	E	K	F	N	P	Y	O	L	J★	G	R	Z	S	I	T	A	V	B	D	W	C	X	U	H	M	q9
q9	D	J	E	M	O	X	N	K	I★	F	Q	Y	R	H	S	Z	U	A	C	V	B	W	T	G	L	P	q10
q10	I	D	L	N	W	M	J	H★	E	P	X	Q	G	R	Y	T	Z	B	U	A	V	S	F	K	O	C	q11
q11	C	K	M	V	L	I	G★	D	O	W	P	F	Q	X	S	Y	A	T	Z	U	R	E	J	N	B	H	q12
q12	J	L	U	K	H	F★	C	N	V	O	E	P	W	R	X	Z	S	Y	T	Q	D	I	M	A	G	B	q13
q13	K	T	J	G	E★	B	M	U	N	D	O	V	Q	W	Y	R	X	S	P	C	H	L	Z	F	A	I	q14
q14	S	I	F	D★	A	L	T	M	C	N	U	P	V	X	Q	W	R	O	B	G	K	Y	E	Z	H	J	q15

STATE	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	NEXT STATE
q15	H	E	C★	Z	K	S	L	B	M	T	O	U	W	P	V	Q	N	A	F	J	X	D	Y	G	I	R	q16
q16	D	B★	Y	J	R	K	A	L	S	N	T	V	O	U	P	M	Z	E	I	W	C	X	F	H	Q	G	q17
q17	A★	X	I	Q	J	Z	K	R	M	S	U	N	T	O	L	Y	D	H	V	B	W	E	G	P	F	C	q18
q18	W	H	P	I	Y	J	Q	L	R	T	M	S	N	K	X	C	G	U	A	V	D	F	O	E	B	Z★	q19
q19	G	O	H	X	I	P	K	Q	S	L	R	M	J	W	B	F	T	Z	U	C	E	N	D	A	Y★	V	q20
q20	N	G	W	H	O	J	P	R	K	Q	L	I	V	A	E	S	Y	T	B	D	M	C	Z	X★	U	F	q21
q21	F	V	G	N	I	O	Q	J	P	K	H	U	Z	D	R	X	S	A	C	L	B	Y	W★	T	E	M	q22
q22	U	F	M	H	N	P	I	O	J	G	T	Y	C	Q	W	R	Z	B	K	A	X	V★	S	D	L	E	q23
q23	E	L	G	M	O	H	N	I	F	S	X	B	P	V	Q	Y	A	J	Z	W	U★	R	C	K	D	T	q24
q24	K	F	L	N	G	M	H	E	R	W	A	O	U	P	X	Z	I	Y	V	T★	Q	B	J	C	S	D	q25
q25	E	K	M	F	L	G	D	Q	V	Z	N	T	O	W	Y	H	X	U	S★	P	A	I	B	R	C	J	q0

Completeness Property: All 26 states × 26 input letters = 676 transitions are completely and deterministically defined. There are no implicit rejects, dead states, or missing transitions, proving that this is a total, complete Mealy machine.

5b. All 676 Transitions in Plain English

Exhaustive catalog of every single transition defined by the machine data, described in plain conversational English.

Note: This printed view shows the currently displayed transition rows. The complete 676-row catalog is available with filters on the live site.

# ▲	FROM	READ	OUTPUT	TO	PLAIN-ENGLISH READING
1	q0	A	J	q1	In q0, key A is pressed: the scrambler outputs J and the rotor moves to q1.
2	q0	B	L	q1	In q0, key B is pressed: the scrambler outputs L and the rotor moves to q1.
3	q0	C	E	q1	In q0, key C is pressed: the scrambler outputs E and the rotor moves to q1.
4	q0	D	K	q1	In q0, key D is pressed: the scrambler outputs K and the rotor moves to q1.
5	q0	E	F	q1	In q0, key E is pressed: the scrambler outputs F and the rotor moves to q1.
6	q0	F	C	q1	In q0, key F is pressed: the scrambler outputs C and the rotor moves to q1.
7	q0	G	P	q1	In q0, key G is pressed: the scrambler outputs P and the rotor moves to q1.

# ▲	FROM	READ	OUTPUT	TO	PLAIN-ENGLISH READING
8	q0	H	U	q1	In q0, key H is pressed: the scrambler outputs U and the rotor moves to q1.
9	q0	I	Y	q1	In q0, key I is pressed: the scrambler outputs Y and the rotor moves to q1.
10	q0	J	M	q1	In q0, key J is pressed: the scrambler outputs M and the rotor moves to q1.
11	q0	K	S	q1	In q0, key K is pressed: the scrambler outputs S and the rotor moves to q1.
12	q0	L	N	q1	In q0, key L is pressed: the scrambler outputs N and the rotor moves to q1.
13	q0	M	V	q1	In q0, key M is pressed: the scrambler outputs V and the rotor moves to q1.
14	q0	N	X	q1	In q0, key N is pressed: the scrambler outputs X and the rotor moves to q1.
15	q0	O	G	q1	In q0, key O is pressed: the scrambler outputs G and the rotor moves to q1.
16	q0	P	W	q1	In q0, key P is pressed: the scrambler outputs W and the rotor moves to q1.
17	q0	Q	T	q1	In q0, key Q is pressed: the scrambler outputs T and the rotor moves to q1.
18	q0	R	R ★	q1	In q0, key R is pressed: the scrambler outputs R and the rotor moves to q1.
19	q0	S	O	q1	In q0, key S is pressed: the scrambler outputs O and the rotor moves to q1.
20	q0	T	Z	q1	In q0, key T is pressed: the scrambler outputs Z and the rotor moves to q1.
21	q0	U	H	q1	In q0, key U is pressed: the scrambler outputs H and the rotor moves to q1.
22	q0	V	A	q1	In q0, key V is pressed: the scrambler outputs A and the rotor moves to q1.

# ▲	FROM	READ	OUTPUT	TO	PLAIN-ENGLISH READING
23	q0	W	Q	q1	In q0, key W is pressed: the scrambler outputs Q and the rotor moves to q1.
24	q0	X	B	q1	In q0, key X is pressed: the scrambler outputs B and the rotor moves to q1.
25	q0	Y	I	q1	In q0, key Y is pressed: the scrambler outputs I and the rotor moves to q1.
26	q0	Z	D	q1	In q0, key Z is pressed: the scrambler outputs D and the rotor moves to q1.

Section 6: State-by-State Details

Each card answers what the state means, how we get here, where we go, and what happens on other input.

Start state (q0)

State q0 ★ Fixed Point: R → R Position 0 Advances to q1 −

MEANING

Rotor position 0, meaning 0 key presses after the initial reference position.

REACHED BY

State q25 on any key, as well as the initial starting reference state of every run.

OUTGOING TRANSITION

Advances to state `q1` on any valid key press. Sample outputs: `A→J, B→L, C→E...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→J	B→L	C→E	D→K	E→F	F→C	G→P	H→U	I→Y
J→M	K→S	L→N	M→V	N→X	O→G	P→W	Q→T	R→R ★
S→O	T→Z	U→H	V→A	W→Q	X→B	Y→I	Z→D	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

Start state of every machine run.

Mid-ring states (q1 to q24)

State q1 ★ Fixed Point: Q → Q

Position 1 Advances to q2 −

MEANING

Rotor position 1, meaning 1 key presses after the initial reference position.

REACHED BY

State q0 on any key.

OUTGOING TRANSITION

Advances to state `q2` on any valid key press. Sample outputs: `A→K, B→D, C→J...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→K	B→D	C→J	D→E	E→B	F→O	G→T	H→X	I→L
J→R	K→M	L→U	M→W	N→F	O→V	P→S	Q→Q ★	R→N
S→Y	T→G	U→Z	V→P	W→A	X→H	Y→C	Z→I	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter Q mapped to itself.

State q2 ★ Fixed Point: P → P

Position 2 Advances to q3

MEANING

Rotor position 2, meaning 2 key presses after the initial reference position.

REACHED BY

State q1 on any key.

OUTGOING TRANSITION

Advances to state `q3` on any valid key press. Sample outputs: `A→C, B→I, C→D...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→C	B→I	C→D	D→A	E→N	F→S	G→W	H→K	I→Q
J→L	K→T	L→V	M→E	N→U	O→R	P→P ★	Q→M	R→X
S→F	T→Y	U→O	V→Z	W→G	X→B	Y→H	Z→J	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter P mapped to itself.

State q3 ★ Fixed Point: O → O

Position 3 Advances to q4

MEANING

Rotor position 3, meaning 3 key presses after the initial reference position.

REACHED BY

State q2 on any key.

OUTGOING TRANSITION

Advances to state `q4` on any valid key press. Sample outputs: `A→H, B→C, C→Z...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→H	B→C	C→Z	D→M	E→R	F→V	G→J	H→P	I→K
J→S	K→U	L→D	M→T	N→Q	O→O ★	P→L	Q→W	R→E
S→X	T→N	U→Y	V→F	W→A	X→G	Y→I	Z→B	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter O mapped to itself.

State q4 ★ Fixed Point: N → N

Position 4 Advances to q5

MEANING

Rotor position 4, meaning 4 key presses after the initial reference position.

REACHED BY

State q3 on any key.

OUTGOING TRANSITION

Advances to state `q5` on any valid key press. Sample outputs: `A→B, B→Y, C→L...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→B	B→Y	C→L	D→Q	E→U	F→I	G→O	H→J	I→R
J→T	K→C	L→S	M→P	N→N ★	O→K	P→V	Q→D	R→W
S→M	T→X	U→E	V→Z	W→F	X→H	Y→A	Z→G	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter N mapped to itself.

State q5 ★ Fixed Point: M → M

Position 5 Advances to q6

MEANING

Rotor position 5, meaning 5 key presses after the initial reference position.

REACHED BY

State q4 on any key.

OUTGOING TRANSITION

Advances to state `q6` on any valid key press. Sample outputs: `A→X, B→K, C→P...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→X	B→K	C→P	D→T	E→H	F→N	G→I	H→Q	I→S
J→B	K→R	L→O	M→M ★	N→J	O→U	P→C	Q→V	R→L
S→W	T→D	U→Y	V→E	W→G	X→Z	Y→F	Z→A	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter M mapped to itself.

State q6 ★ Fixed Point: L → L

Position 6 Advances to q7

MEANING

Rotor position 6, meaning 6 key presses after the initial reference position.

REACHED BY

State q5 on any key.

OUTGOING TRANSITION

Advances to state `q7` on any valid key press. Sample outputs: `A→J, B→O, C→S...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→J	B→O	C→S	D→G	E→M	F→H	G→P	H→R	I→A
J→Q	K→N	L→L ★	M→I	N→T	O→B	P→U	Q→K	R→V
S→C	T→X	U→D	V→F	W→Y	X→E	Y→Z	Z→W	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter L mapped to itself.

State q7 ★ Fixed Point: K → K

Position 7 Advances to q8

MEANING

Rotor position 7, meaning 7 key presses after the initial reference position.

REACHED BY

State q6 on any key.

OUTGOING TRANSITION

Advances to state `q8` on any valid key press. Sample outputs: `A→N, B→R, C→F...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→N	B→R	C→F	D→L	E→G	F→O	G→Q	H→Z	I→P
J→M	K→K ★	L→H	M→S	N→A	O→T	P→J	Q→U	R→B
S→W	T→C	U→E	V→X	W→D	X→Y	Y→V	Z→I	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter K mapped to itself.

State q8 ★ Fixed Point: J → J

Position 8 Advances to q9

MEANING

Rotor position 8, meaning 8 key presses after the initial reference position.

REACHED BY

State q7 on any key.

OUTGOING TRANSITION

Advances to state `q9` on any valid key press. Sample outputs: `A→Q, B→E, C→K...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→Q	B→E	C→K	D→F	E→N	F→P	G→Y	H→O	I→L
J→J ★	K→G	L→R	M→Z	N→S	O→I	P→T	Q→A	R→V
S→B	T→D	U→W	V→C	W→X	X→U	Y→H	Z→M	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter J mapped to itself.

State q9 ★ Fixed Point: I → I

Position 9 Advances to q10

MEANING

Rotor position 9, meaning 9 key presses after the initial reference position.

REACHED BY

State q8 on any key.

OUTGOING TRANSITION

Advances to state `q10` on any valid key press. Sample outputs: `A→D, B→J, C→E...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→D	B→J	C→E	D→M	E→O	F→X	G→N	H→K	I→I ★
J→F	K→Q	L→Y	M→R	N→H	O→S	P→Z	Q→U	R→A
S→C	T→V	U→B	V→W	W→T	X→G	Y→L	Z→P	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter I mapped to itself.

State q10

★ Fixed Point: H → H

Position 10 Advances to q11

MEANING

Rotor position 10, meaning 10 key presses after the initial reference position.

REACHED BY

State q9 on any key.

OUTGOING TRANSITION

Advances to state `q11` on any valid key press. Sample outputs: `A→I, B→D, C→L...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→I	B→D	C→L	D→N	E→W	F→M	G→J	H→H ★	I→E
J→P	K→X	L→Q	M→G	N→R	O→Y	P→T	Q→Z	R→B
S→U	T→A	U→V	V→S	W→F	X→K	Y→O	Z→C	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter H mapped to itself.

State q11 ★ Fixed Point: G → G

Position 11 Advances to q12

MEANING

Rotor position 11, meaning 11 key presses after the initial reference position.

REACHED BY

State q10 on any key.

OUTGOING TRANSITION

Advances to state `q12` on any valid key press. Sample outputs: `A→C, B→K, C→M...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→C	B→K	C→M	D→V	E→L	F→I	G→G ★	H→D	I→O
J→W	K→P	L→F	M→Q	N→X	O→S	P→Y	Q→A	R→T
S→Z	T→U	U→R	V→E	W→J	X→N	Y→B	Z→H	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter G mapped to itself.

State q12

★ Fixed Point: F → F

Position 12 Advances to q13

MEANING

Rotor position 12, meaning 12 key presses after the initial reference position.

REACHED BY

State q11 on any key.

OUTGOING TRANSITION

Advances to state `q13` on any valid key press. Sample outputs: `A→J, B→L, C→U...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→J	B→L	C→U	D→K	E→H	F→F ★	G→C	H→N	I→V
J→O	K→E	L→P	M→W	N→R	O→X	P→Z	Q→S	R→Y
S→T	T→Q	U→D	V→I	W→M	X→A	Y→G	Z→B	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter F mapped to itself.

State q13 ★ Fixed Point: E → E

Position 13 Advances to q14

MEANING

Rotor position 13, meaning 13 key presses after the initial reference position.

REACHED BY

State q12 on any key.

OUTGOING TRANSITION

Advances to state `q14` on any valid key press. Sample outputs: `A→K, B→T, C→J...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→K	B→T	C→J	D→G	E→E ★	F→B	G→M	H→U	I→N
J→D	K→O	L→V	M→Q	N→W	O→Y	P→R	Q→X	R→S
S→P	T→C	U→H	V→L	W→Z	X→F	Y→A	Z→I	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter E mapped to itself.

State q14 ★ Fixed Point: D → D

Position 14 Advances to q15

MEANING

Rotor position 14, meaning 14 key presses after the initial reference position.

REACHED BY

State q13 on any key.

OUTGOING TRANSITION

Advances to state `q15` on any valid key press. Sample outputs: `A→S, B→I, C→F...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→S	B→I	C→F	D→D ★	E→A	F→L	G→T	H→M	I→C
J→N	K→U	L→P	M→V	N→X	O→Q	P→W	Q→R	R→O
S→B	T→G	U→K	V→Y	W→E	X→Z	Y→H	Z→J	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter D mapped to itself.

State q15

★ Fixed Point: C → C

Position 15 Advances to q16

MEANING

Rotor position 15, meaning 15 key presses after the initial reference position.

REACHED BY

State q14 on any key.

OUTGOING TRANSITION

Advances to state `q16` on any valid key press. Sample outputs: `A→H, B→E, C→C...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→H	B→E	C→C ★	D→Z	E→K	F→S	G→L	H→B	I→M
J→T	K→O	L→U	M→W	N→P	O→V	P→Q	Q→N	R→A
S→F	T→J	U→X	V→D	W→Y	X→G	Y→I	Z→R	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter C mapped to itself.

State q16 ★ Fixed Point: B → B

Position 16 Advances to q17

MEANING

Rotor position 16, meaning 16 key presses after the initial reference position.

REACHED BY

State q15 on any key.

OUTGOING TRANSITION

Advances to state `q17` on any valid key press. Sample outputs: `A→D, B→B, C→Y...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→D	B→B ★	C→Y	D→J	E→R	F→K	G→A	H→L	I→S
J→N	K→T	L→V	M→O	N→U	O→P	P→M	Q→Z	R→E
S→I	T→W	U→C	V→X	W→F	X→H	Y→Q	Z→G	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter B mapped to itself.

State q17

★ Fixed Point: A → A

Position 17 Advances to q18

MEANING

Rotor position 17, meaning 17 key presses after the initial reference position.

REACHED BY

State q16 on any key.

OUTGOING TRANSITION

Advances to state `q18` on any valid key press. Sample outputs: `A→A, B→X, C→I...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→A ★	B→X	C→I	D→Q	E→J	F→Z	G→K	H→R	I→M
J→S	K→U	L→N	M→T	N→O	O→L	P→Y	Q→D	R→H
S→V	T→B	U→W	V→E	W→G	X→P	Y→F	Z→C	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter A mapped to itself.

State q18 ★ Fixed Point: Z → Z

Position 18 Advances to q19 −

MEANING

Rotor position 18, meaning 18 key presses after the initial reference position.

REACHED BY

State q17 on any key.

OUTGOING TRANSITION

Advances to state `q19` on any valid key press. Sample outputs: `A→W, B→H, C→P...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→W	B→H	C→P	D→I	E→Y	F→J	G→Q	H→L	I→R
J→T	K→M	L→S	M→N	N→K	O→X	P→C	Q→G	R→U
S→A	T→V	U→D	V→F	W→O	X→E	Y→B	Z→Z ★	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter Z mapped to itself.

State q19

★ Fixed Point: Y → Y

Position 19 Advances to q20

MEANING

Rotor position 19, meaning 19 key presses after the initial reference position.

REACHED BY

State q18 on any key.

OUTGOING TRANSITION

Advances to state `q20` on any valid key press. Sample outputs: `A→G, B→O, C→H...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→G	B→O	C→H	D→X	E→I	F→P	G→K	H→Q	I→S
J→L	K→R	L→M	M→J	N→W	O→B	P→F	Q→T	R→Z
S→U	T→C	U→E	V→N	W→D	X→A	Y→Y ★	Z→V	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter Y mapped to itself.

State q20 ★ Fixed Point: X → X

Position 20 Advances to q21

MEANING

Rotor position 20, meaning 20 key presses after the initial reference position.

REACHED BY

State q19 on any key.

OUTGOING TRANSITION

Advances to state `q21` on any valid key press. Sample outputs: `A→N, B→G, C→W...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→N	B→G	C→W	D→H	E→O	F→J	G→P	H→R	I→K
J→Q	K→L	L→I	M→V	N→A	O→E	P→S	Q→Y	R→T
S→B	T→D	U→M	V→C	W→Z	X→X ★	Y→U	Z→F	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter X mapped to itself.

State q21 ★ Fixed Point: W → W

Position 21 Advances to q22

MEANING

Rotor position 21, meaning 21 key presses after the initial reference position.

REACHED BY

State q20 on any key.

OUTGOING TRANSITION

Advances to state `q22` on any valid key press. Sample outputs: `A→F, B→V, C→G...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→F	B→V	C→G	D→N	E→I	F→O	G→Q	H→J	I→P
J→K	K→H	L→U	M→Z	N→D	O→R	P→X	Q→S	R→A
S→C	T→L	U→B	V→Y	W→W ★	X→T	Y→E	Z→M	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter W mapped to itself.

State q22 ★ Fixed Point: V → V

Position 22 Advances to q23

MEANING

Rotor position 22, meaning 22 key presses after the initial reference position.

REACHED BY

State q21 on any key.

OUTGOING TRANSITION

Advances to state `q23` on any valid key press. Sample outputs: `A→U, B→F, C→M...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→U	B→F	C→M	D→H	E→N	F→P	G→I	H→O	I→J
J→G	K→T	L→Y	M→C	N→Q	O→W	P→R	Q→Z	R→B
S→K	T→A	U→X	V→V ★	W→S	X→D	Y→L	Z→E	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter V mapped to itself.

State q23 ★ Fixed Point: U → U

Position 23 Advances to q24

MEANING

Rotor position 23, meaning 23 key presses after the initial reference position.

REACHED BY

State q22 on any key.

OUTGOING TRANSITION

Advances to state `q24` on any valid key press. Sample outputs: `A→E, B→L, C→G...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→E	B→L	C→G	D→M	E→O	F→H	G→N	H→I	I→F
J→S	K→X	L→B	M→P	N→V	O→Q	P→Y	Q→A	R→J
S→Z	T→W	U→U ★	V→R	W→C	X→K	Y→D	Z→T	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter U mapped to itself.

State q24 ★ Fixed Point: T → T

Position 24 Advances to q25

MEANING

Rotor position 24, meaning 24 key presses after the initial reference position.

REACHED BY

State q23 on any key.

OUTGOING TRANSITION

Advances to state `q25` on any valid key press. Sample outputs: `A→K, B→F, C→L...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→K	B→F	C→L	D→N	E→G	F→M	G→H	H→E	I→R
J→W	K→A	L→O	M→U	N→P	O→X	P→Z	Q→I	R→Y
S→V	T→T ★	U→Q	V→B	W→J	X→C	Y→S	Z→D	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

One rotational position of the 26-state cycle, with fixed-point letter T mapped to itself.

Wrap-around state (q25)

State q25 ★ Fixed Point: S → S Position 25 Advances to q0 −

MEANING

Rotor position 25, meaning 25 key presses after the initial reference position.

REACHED BY

State q24 on any key.

OUTGOING TRANSITION

Advances to state `q0` on any valid key press. Sample outputs: `A→E, B→K, C→M...`

OUTPUT PERMUTATION ROW (ALL 26 INPUT/OUTPUT PAIRS)

A→E	B→K	C→M	D→F	E→L	F→G	G→D	H→Q	I→V
J→Z	K→N	L→T	M→O	N→W	O→Y	P→H	Q→X	R→U
S→S ★	T→P	U→A	V→I	W→B	X→R	Y→C	Z→J	

ANYTHING ELSE (HANDLING OTHER INPUTS)

The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves. For every valid key A to Z there is a deterministic move, so there are no reject or dead states.

THEORETICAL NOTE

Next state wraps cyclically to q0, completing a 26-keystroke full revolution.

Section 7: What the Machine Computes

Because a Mealy machine is an information-preserving *transducer* rather than a language acceptor, we do not describe a "language accepted" (which would be trivial: Σ^*). Instead, we formally characterize the mathematical function $f: \Sigma^* \to \Gamma^*$ that it computes: a finite-state transducer that reads a finite input string and emits one output letter per input letter, with no accepting states. A short run, including an empty one, is valid.

Eight Core Computational Properties

1. **Complete:** All 676 possible `(state, letter)` pairs are defined; no valid uppercase input is ever rejected or blocked.
2. **Deterministic:** For every state and input character, there is exactly one next state and one output symbol.
3. **Per-state bijection:** In every state, the mapping across all 26 input letters produces a permutation of `A–Z` with no duplicates and none omitted.
4. **Not self-inverse:** Unlike a complete historical Enigma machine, this isolated rotor lacks a reflector; running the ciphertext through the machine a second time does not recover plaintext.
5. **Fixed points:** In every state, exactly one letter is mapped to itself due to the internal pin wiring offset.
6. **Position matters:** Polyalphabetic substitution ensures that repeated input letters map to different outputs as the rotor advances (e.g. `AAA` from `q0` becomes `JKC`).

7. **Period 26:** Stepping advances modulo 26, meaning the state sequence repeats cyclically every 26 keystrokes (q0 → q1 → … → q25 → q0).
8. **All 26 states are distinguishable:** No two states share identical output substitution rows; thus the machine is minimal and cannot be reduced to fewer states.

Fixed Points by State (Letters Mapped to Themselves)

Dynamically computed from machine transitions where $\lambda(q_N, \sigma) = \sigma$:

q0 → R	q1 → Q	q2 → P	q3 → O	q4 → N	q5 → M	q6 → L	q7 → K	q8 → J
q9 → I	q10 → H	q11 → G	q12 → F	q13 → E	q14 → D	q15 → C	q16 → B	q17 → A
q18 → Z	q19 → Y	q20 → X	q21 → W	q22 → V	q23 → U	q24 → T	q25 → S	

A strength and a weakness, say both

Strength: Mathematical precision and absolute explainability—the transducer provides a deterministic, zero-ambiguity trace for every letter typed.

Weakness: Because this is only a single isolated rotor without a reflector or plugboard, it provides negligible cryptographic security and cannot decrypt reciprocally by re-running ciphertext. The model in this project contains no window letters, no notch positions, and no ring settings; the natural theoretical extension studied in cryptanalysis is cascading three rotors with turnover notches, a reflector, and a permutation plugboard.

Section 8: Simulator and Worked Traces

An interactive step-by-step transducer simulator with an automated text cleaner, bi-directional diagram and matrix synchronization, and an exhaustive live verification suite of 16 worked test traces.

8a. Single Rotor Mealy Machine Simulator

Feed messages into the single rotor. Watch the rotor step deterministically, emit output letters, and cycle through its 26 internal states.

Simulation Input & Configuration

Input Message:

e.g. HELLOWORLD

Start State (q0–q25):

q0 (Rotor pos 0)

☑ Clean my text first (uppercase, remove spaces, digits and punctuation)

Cleaned input: `(empty)` | **Removed:** 0 characters

Execution Step Trace: (Focus table to navigate with ◄ / ► arrows)

STEP	CURRENT STATE	KEY READ	OUTPUT PRODUCED	NEXT STATE
No execution steps to show (enter a message above).				

Simulation Tape & Output

Final state: q0

IN: *No input letters*

OUT: *No output letters*

Full Output String: `(empty)`

Two kinds of outcome

Valid runs: A valid run always ends in some state with an output string, because the machine is complete—every state has a deterministic transition for all 26 uppercase letters A through Z.

Input errors: The machine's alphabet is strictly A to Z, so other characters are not inputs to it. An input error only happens when text outside A to Z is fed in, and it is caught by the input check before any machine move is made for that character. There are no reject or dead states inside the Mealy machine.

8b. Worked Test Traces & Verification Suite

The table below executes 16 canonical test cases (13 valid runs and 3 input errors) directly against the `runEnigmaMachine` simulator engine. Traces 1 to 16 mirror the project's automated test suite, evaluated live on page load.

Live Suite Status: 16 / 16 match

#	START STATE	INPUT	EXPECTED OUTPUT	EXPECTED FINAL	RESULT	EXECUTION PATH	NOTE
1	`q0`	`A`	`J`	`q1`	✓ Match	`q0 → q1`	The first key press.
2	`q0`	`AAA`	`JKC`	`q3`	✓ Match	`q0 → q1 → q2 → q3`	The same key gives different outputs as the rotor moves.

#	START STATE	INPUT	EXPECTED OUTPUT	EXPECTED FINAL	RESULT	EXECUTION PATH	NOTE
3	q0	ABC	JDD	q3	✓ Match	q0 → q1 → q2 → q3	The example used in Section 2.
4	q0	ZZZ	DIJ	q3	✓ Match	q0 → q1 → q2 → q3	Three consecutive Z inputs.
5	q0	BBBB	LDIC	q4	✓ Match	q0 → q1 → q2 → q3 → q4	Four identical inputs.
6	q5	M	M	q6	✓ Match	q5 → q6	A fixed point, the only letter in q5 that maps to itself.
7	q25	AA	EJ	q1	✓ Match	q25 → q0 → q1	Wrap-around from q25 to q0.
8	q25	XYZ	RII	q2	✓ Match	q25 → q0 → q1 → q2	Wrap-around with three keys.
9	q0	AAAAAAAAAA... (26×A)	JKCHBXJNQDICJKSHDAWGNFUEKE	q0	✓ Match	q0 → q1 → q2 → q3 → q4 → q5 → q6 → q7 → q8 → q9 → q10 → q11 → q12 → q13 → q14 → q15 → q16 → q17 → q18 → q19 → q20 → q21 → q22 → q23 → q24 → q25 → q0	A full turn returns to the start state.
10	q0	HELLOWORLD	UBVDKGBBRM	q10	✓ Match	q0 → q1 → q2 → q3 → q4 → q5 → q6 → q7 → q8 → q9 → q10	Ten-letter standard message.
11	q0	ATTACKATDAWN	JGYHLRJCFDFX	q12	✓ Match	q0 → q1 → q2 → q3 → q4 → q5 → q6 → q7 → q8 → q9 → q10 → q11 → q12	Historical 12-letter test phrase.
12	q10	HELLO	HLPVQ	q15	✓ Match	q10 → q11 → q12 → q13 → q14 → q15	A run starting mid-cycle.
13	q0	*(empty)*	*(empty)*	q0	✓ Match	q0	An empty run stays in the start state.
14	q0	a	error	q0	✓ Match	q0 ✸ (halted: 'a' at pos 1)	Expected error naming 'a' at position 1.
15	q0	A1	error	q1	✓ Match	q0 → q1 ✸ (halted: '1' at pos 2)	Expected error naming '1' at position 2; trace shows q0 then q1 before failure.
16	q0	HELLO WORLD	error	q5	✓ Match	q0 → q1 → q2 → q3 → q4 → q5 ✸ (halted: space at pos 6)	Expected error naming the space at position 6.

Two kinds of outcome

Valid inputs: Always produce an output string and a final state. Because δ is total on $\Sigma = \{A, \ldots, Z\}$, every valid character advances the rotor position.

Invalid characters: The machine's alphabet is strictly A to Z, so other characters are not inputs to it. They are intercepted by the input check before the machine moves, so there are no reject or dead states in Q.

Section 9: Theory Points a Teacher May Probe

Key theoretical distinctions and viva questions on finite automata and computability theory:

1. Is it really a Mealy machine?

Yes, it strictly satisfies the mathematical definition of a Mealy machine. It possesses a finite set of 26 states Q, finite input and output alphabets $\Sigma = \Gamma = \{A, \ldots, Z\}$, a total state transition function $\delta: Q \times \Sigma \to Q$, an output function $\lambda: Q \times \Sigma \to \Gamma$ whose values depend directly on both current state and input symbol, and a designated start state q0.

2. Complete vs. partial machine

Our machine is completely defined: every one of the 26 × 26 = 676 possible state-input pairs has an explicit transition and output. The machine's input alphabet is strictly $\Sigma = \{A, \ldots, Z\}$, so characters outside this set are not inputs to it and are refused by the input check before the machine moves. Because every valid key A to Z deterministically advances the rotor, the transition function δ is total, with no undefined moves, no reject states, and no dead states.

3. Minimisation and state equivalence

Using the standard table-filling equivalence partition algorithm, two states p and q are equivalent ($p \equiv q$) if and only if for all input strings $w \in \Sigma^*$, $\hat{\lambda}(p, w) = \hat{\lambda}(q, w)$. Since every state has a distinct 1-length output vector across the 26 input letters, no two states are 1-equivalent. Consequently, all 26 states are pairwise distinguishable, proving that the machine is already minimal and cannot be reduced.

4. Mealy vs. Moore vs. DFA

In a Mealy machine, outputs are associated with transitions ($\lambda: Q \times \Sigma \to \Gamma$), allowing immediate response to input keys. In a Moore machine, outputs are tied solely to states ($\lambda: Q \to \Gamma$); any Mealy machine can be converted to an equivalent Moore machine, but doing so generally inflates the state count (up to $|Q| \times |\Gamma|$ states). A DFA produces no stream of outputs at all, providing only a binary accept/reject verdict at the end of input.

5. Computational complexity

Simulation requires O(n) time complexity for an input string of length n, because each character requires a single O(1) constant-time transition table lookup. Memory complexity for execution is O(1) auxiliary space, since only the single scalar state variable must be maintained. When storing the full diagnostic trace of all visited states and step records, the trace buffer consumes O(n) space.

6. What it cannot do (Theoretical limitations)

Because finite state automata lack unbounded external memory (like a stack or Turing tape), this machine cannot recognize non-regular languages such as $a^n b^n$ or count unboundedly. Furthermore, it cannot perform cryptanalysis or simulate the Turing Bombe search spaces (2,109,120 combinations for German Army/Air Force Enigma, or 11,811,072 for Navy). Those cryptanalytic searches are finite combinatorial brute-force problems whose difficulty lies in exponential search time rather than computational undecidability.

Section 10: Implementation Details

One source of truth for the machine

All machine states, alphabet definitions, the initial state, and all 676 transition cells are declared exclusively in `enigmaData.js`. The web application never hard-codes machine constants; every stat tile, trace, and table across this study guide dynamically queries this single source of truth at page load.

How the implementation works

The system is constructed with zero frameworks, zero bundlers, and zero external runtime dependencies:

- **`enigmaData.js`**: Contains the single source-of-truth JavaScript object defining states, alphabets, start state, and the complete 26 × 26 transition lookup table.
- **`validator.js`**: Implements automated assertion checks that verify state counts, completeness (676 transitions), cyclic stepping $q_N \to q_{N+1}$, and output bijectivity.
- **`enigmaMachine.js`**: Contains the pure simulation function `runEnigmaMachine` that evaluates input strings without side effects or global mutations.
- **`test.js`**: Automated Node.js test suite running 19 exhaustive test cases covering rotor stepping, wrap-around, full cycles, and character validation.
- **`index.html`**: Semantic, accessible HTML5 structure providing the complete study guide interface across all 11 sections (Sections 0 through 10).
- **`style.css`**: Pure vanilla CSS implementing responsive typography, high-contrast academic aesthetics, and print media rules.

- `app.js` : Lightweight vanilla browser script that reads machine data, calculates live traces, populates DOM elements, powers the interactive state diagram, transition matrix, state profiles, the step-by-step interactive simulator with text cleaning and auto-play, and the live worked traces verification table, and executes 18 automated startup self-checks in the browser console.

How to run the tests

To run the automated verification suite offline, open a terminal in the project directory and execute:

```
node test.js
```

The terminal executes the 4-rule data validator and all 18 simulation test cases in `test.js`, concluding with `TEST SUMMARY: 19/19 match`, while the browser console executes 15 automated runtime self-checks, concluding with `All System Self-Checks PASSED (15/15)`.

Simulation Function Return Structure

The pure execution function `runEnigmaMachine` takes the machine data, an input string, and an optional start state (defaulting to `q0`). It returns a structured result object containing:

- `output` : The resulting ciphertext string formed by concatenating each emitted character.
- `visitedStates` : The ordered array of all states visited during execution, beginning with the start state.
- `finalState` : The state where the rotor rests after processing the final input character.
- `steps` : An array of step objects, each recording the active state, input letter read, output letter generated, and destination state.
- `error` : A descriptive error message detailing the offending character and its 1-based position if an invalid token is encountered, or an empty string for valid runs.

A Mealy Machine Model of the Enigma Rotor

Models of Computation (Research Paper Review)

Team Members: Harsha Narayan J, Lingeshwar Rethinam, Sanjay, Pavan Aadarsh

Academic Reference:

Downs, A. S., Sigmon, N. P., & Klima, R. E. *Cryptanalysis of the Enigma Machine: The Bombe and Beyond*. The Electronic Journal of Mathematics and Technology, 19(2).

📄 **Read Research Paper (PDF, opens in new tab)**

Built with Plain HTML, CSS & Vanilla JavaScript. Zero External Runtime Dependencies.